ⵜⴰⵖⵣⵓⵍⵜ ⵏ ⵜⵙⴰⵡⴰⵍⵜ
FACULTÉ DES SCIENCES

MARRAKECH
جامعة القاضي عياض
UNIVERSITÉ CADI AYYAD

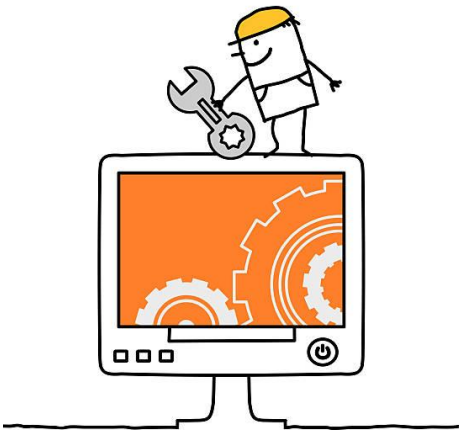
Semlalia

كليـــــــــة العلوم
FACULTY OF SCIENCE

Rapport de projet de fin du module Fondements avancés de l'IA

Intitulé

Système expert de diagnostic des pannes d'ordinateurs



Réalisé par : Jihane HAMOUCHI et Noura RIFAFI

Encadré par : Pr. EL ALAOUI HASNA

Master 1 intelligence artificielle

Année universitaire : 2025 / 2026

Table des matières

I. Problématique 5

II. Introduction générale 6

1. Contexte du projet 6

2. Objectifs du projet 6

3. Présentation générale des systèmes experts 7

III. Analyse et modélisation du problème 8

1. Base de connaissances utilisée pour l'identification des pannes 8

- Problèmes d'alimentation 9
- Problèmes liés au système de refroidissement 9
- Problèmes d'affichage 9
- Problèmes de disque dur/SSD 10
- Problèmes de mémoire vive (RAM) 10
- Problèmes périphériques 10
- Problèmes carte mère 10
- Problèmes réseau 11
- Problèmes logiciels 11
- Problèmes système d'exploitation 11
- Problèmes audios 12

2. Détermination des solutions correctives fondées sur la base de connaissances 12

- Solution pour problèmes d'alimentation 12
- Solution pour problèmes de refroidissement 12
- Solution pour problèmes d'affichage 13
- Solution pour problèmes disque dur / SSD 13
- Solution pour problèmes de mémoire RAM 13
- Solution pour problèmes périphériques 13

- Solution pour problèmes carte mère 13
- Solution pour problèmes réseau 13
- Solution pour problèmes logiciels 14
- Solution pour problèmes du système d'exploitation 14
- Solution pour problèmes audio 14

IV. Réalisation du système expert 15

1. Choix des technologies 15
   - Prolog 15
   - Tkinter 15
   - Communication entre Prolog et tkinter 16
2. Implémentation de la base de connaissances en Prolog 16
   - Gestion dynamique des symptômes 16
   - Structure des règles de diagnostic 17
   - Gestion des pannes partielles 18
   - Encodage des solutions correctives 18
   - Moteur d'inférence et logique de raisonnement 18
   - Fonctions utilitaires 19
3. Interface IHM avec Tkinter 19
   - Initialisation et connexion au moteur Prolog 19
   - Organisation des symptômes 19
   - Création de la fenêtre principale 20
   - Génération dynamique des catégories et symptômes 20
   - Collecte des symptômes et diagnostic 20
   - Affichage du résultat 21
   - Réinitialisation 21

V. Résultats obtenus 22

1. Fonctionnement général 22

2. Exemples de résultats 23
3. Performance 24
VI. Conclusion 25
Webographie : 26

I. Problématique

La complexité croissante des systèmes informatiques modernes, résultant de l'intégration simultanée de composants matériels, logiciels et systèmes d'exploitation de plus en plus sophistiqués, rend le diagnostic des pannes particulièrement délicat pour les utilisateurs non experts. Les méthodes classiques de dépannage reposent souvent sur l'intervention d'un technicien spécialisé ou sur des procédures empiriques, pouvant entraîner des délais, des coûts élevés et des risques d'erreur.

Dans ce contexte, la problématique centrale de ce travail est la suivante :

- **Comment concevoir un système intelligent, fondé sur les principes de l'intelligence artificielle symbolique, capable de formaliser et d'exploiter l'expertise humaine afin de diagnostiquer de manière fiable et cohérente des pannes d'ordinateurs à partir de symptômes observables, tout en garantissant la transparence du raisonnement et la facilité d'utilisation du système ?**



II. Introduction générale

1. Contexte du projet

L'essor des technologies informatiques a conduit à la multiplication et à la diversification des composants matériels et logiciels, rendant la maintenance et le diagnostic des systèmes plus complexes et plus exigeants. Les utilisateurs non experts se trouvent souvent démunis face à des pannes dont l'origine peut être multiple : défaillance matérielle, problème logiciel, conflit entre applications ou erreur de configuration. La nécessité d'une assistance rapide et fiable est donc un enjeu crucial, tant pour la productivité que pour la sécurité des données.

Les systèmes experts, en tant qu'outil d'intelligence artificielle symbolique, constituent une réponse adaptée à cette problématique. Ils permettent de formaliser l'expertise humaine en connaissances explicites et d'utiliser des mécanismes d'inférence logique pour reproduire un raisonnement cohérent et structuré. L'automatisation partielle du diagnostic peut ainsi améliorer la réactivité du processus, réduire les coûts liés aux interventions humaines et offrir une solution d'assistance accessible aux utilisateurs, tout en garantissant la traçabilité et l'explicabilité du raisonnement fourni.

Dans ce contexte, ce projet vise à explorer et à mettre en œuvre un système expert capable d'assister un utilisateur dans l'identification et la compréhension des pannes informatiques.

2. Objectifs du projet

Les objectifs de ce projet peuvent être articulés autour de plusieurs axes complémentaires :

- **Formalisation des connaissances** : identifier, structurer et représenter les connaissances nécessaires au diagnostic des pannes informatiques afin qu'elles soient exploitables par un système intelligent.
- **Développement du moteur de raisonnement** : concevoir et mettre en œuvre un moteur capable d'utiliser ces connaissances pour fournir des diagnostics fiables et cohérents, en reproduisant le raisonnement d'un expert humain.
- **Conception de l'interface utilisateur** : créer une interface ergonomique et intuitive permettant de collecter les symptômes auprès de l'utilisateur et de présenter les diagnostics de manière claire et compréhensible.
- **Validation et évaluation** : tester le système à travers différents scénarios réalistes afin d'évaluer sa pertinence, sa fiabilité et la cohérence de ses résultats.

- **Analyse critique et perspectives** : identifier les limites du système développé, proposer des améliorations possibles et envisager des extensions pour une couverture plus large des types de pannes ou une meilleure intégration dans des environnements réels.

3. Présentation générale des systèmes experts

Un système expert est un programme informatique conçu pour simuler le raisonnement d'un expert humain dans un domaine spécifique, en s'appuyant sur une base de connaissances structurée et un moteur d'inférence capable de traiter les informations et de produire des solutions ou diagnostics cohérents. Il se compose essentiellement de trois éléments **: la base de connaissances**, qui stocke les faits et les règles définissant l'expertise du domaine ; **le moteur d'inférence**, qui applique des mécanismes logiques pour déduire des conclusions à partir des données disponibles ; et **l'interface utilisateur**, qui permet la communication avec le système en assurant la saisie des informations et la restitution des résultats de manière claire et compréhensible. L'intelligence artificielle symbolique joue un rôle central dans ces systèmes en permettant la représentation explicite des connaissances et le raisonnement logique, offrant ainsi la possibilité de produire des diagnostics explicables et fiables. Les systèmes experts présentent plusieurs avantages, notamment la capacité à reproduire un raisonnement expert, à fournir des solutions cohérentes et à réduire la dépendance à l'expertise humaine directe, mais ils comportent également des limites, telles que la dépendance à la qualité et à l'exhaustivité de la base de connaissances, la rigidité face à des situations non prévues et la difficulté à gérer l'incertitude ou des informations partielles. Cette approche constitue néanmoins un cadre efficace pour automatiser le diagnostic de problèmes complexes tout en garantissant la transparence et la traçabilité du raisonnement.

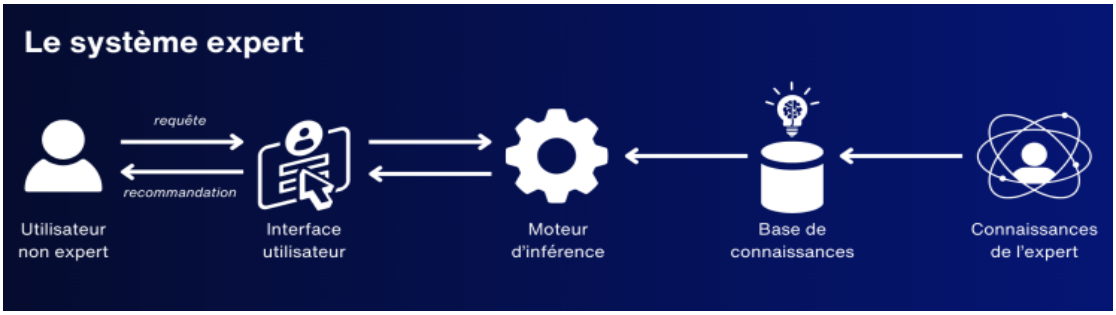


Figure 1 Illustration du fonctionnement des systèmes expert

III. Analyse et modélisation du problème

La conception d'un système expert de diagnostic de pannes informatiques nécessite une compréhension approfondie du domaine et une formalisation rigoureuse des connaissances. Cette étape commence par l'étude des types de pannes les plus fréquentes, qu'elles soient d'origine matérielle ou logicielle, et par l'identification des symptômes qui leur sont associés, tels que messages d'erreur, performances anormales ou impossibilité de démarrage. À partir de ces observations, les relations causales entre symptômes et pannes sont définies afin de structurer un modèle logique permettant au système d'inférer des diagnostics cohérents et explicables. La formalisation des connaissances consiste à représenter chaque panne et ses symptômes sous forme de règles logiques exploitables par le moteur d'inférence, tout en tenant compte des informations partielles ou ambiguës. Enfin, un périmètre fonctionnel précis est établi pour limiter la complexité du système et garantir la pertinence des diagnostics, en incluant uniquement les pannes pour lesquelles des symptômes fiables ont pu être identifiés. Cette démarche méthodologique assure ainsi une modélisation claire, structurée et extensible, essentielle pour la conception et l'évaluation d'un système expert performant.

1. Base de connaissances utilisée pour l'identification des pannes

Pour élaborer la base de connaissances de notre système expert, nous nous sommes appuyés sur le site spécialisé *Docteur Ordinateur*, qui recense les 12 pannes les plus courantes rencontrées par les utilisateurs d'ordinateurs. Cette ressource fournit une vision complète des dysfonctionnements possibles, couvrant à la fois les problèmes matériels, tels que les défaillances d'alimentation, de mémoire vive, de disque dur ou de composants centraux, et les problèmes logiciels, incluant les plantages d'applications, les erreurs système ou les anomalies du système d'exploitation. Elle décrit également les symptômes associés à chaque panne, allant des messages d'erreur et écrans noirs aux ralentissements, aux redémarrages intempestifs, aux problèmes de périphériques ou aux difficultés de connectivité. Ces informations ont permis d'identifier les relations causales entre symptômes et pannes et de formaliser les connaissances sous forme de règles logiques, constituant ainsi le fondement du moteur d'inférence du système expert. Cette structuration assure la capacité du système à relier de manière cohérente les observations de l'utilisateur à des diagnostics plausibles et explicables, tout en offrant une base extensible pour intégrer de nouvelles pannes ou situations futures. À partir du site nous avons identifié les pannes et leurs symptômes caractéristiques comme suit :

- Problèmes d'alimentation

Cette classe comprend plusieurs problèmes : alimentation défaillante, extinction soudaine, bloc d'alimentation faible. Tous ces problèmes touchent l'énergie fournie à l'ordinateur, mais chacun a des symptômes légèrement différents.

- **Alimentation défaillante** : l'ordinateur ne démarre pas, aucun bruit du ventilateur n'est perceptible, et aucune LED ne s'allume.
- **Bloc d'alimentation faible** : l'ordinateur ne démarre pas et aucune LED ne s'allume, indiquant une puissance insuffisante.
- **Extinction soudaine** : l'ordinateur s'éteint brutalement et peut être très chaud au toucher.

- Problèmes liés au système de refroidissement

Plusieurs pannes sont regroupées ici : surchauffe du processeur, surchauffe générale, ventilateur défectueux ou bruyant, problème de ventilation. Elles ont toutes pour point commun la chaleur excessive, mais certaines concernent le ventilateur, d'autres le processeur ou le système global.

- **Surchauffe du processeur** : l'ordinateur est très chaud, des ralentissements fréquents sont observés, et des arrêts inopinés peuvent survenir.
- **Surchauffe générale** : l'ordinateur devient très lent et instable.
- **Ventilateur défectueux** : bruits anormaux du ventilateur accompagnés d'une forte chaleur de l'ordinateur.
- **Ventilateur bruyant** : le ventilateur produit des bruits inhabituels, même si l'ordinateur continue de fonctionner.
- **Problème de ventilation** : combinaison de surchauffe et de bruit du ventilateur indiquant une circulation d'air insuffisante.

- Problèmes d'affichage

Cette catégorie inclut carte graphique défectueuse, problème d'affichage, et écran défectueux. Tous entraînent des anomalies visuelles, mais la cause exacte varie.

- **Carte graphique défectueuse** : l'écran reste noir au démarrage, mais l'ordinateur émet des sons de démarrage, et le câble n'est pas en cause.
- **Problème d'affichage** : écran noir au démarrage, sans distinction précise de la cause.
- **Écran défectueux** : présence de pixels morts et déformation des couleurs.

- Problèmes de disque dur/SSD

On y trouve disque dur défaillant, disque dur critique, disque saturé, erreur disque. La panne affecte le stockage et la vitesse d'accès aux fichiers, mais les symptômes (bruits, lenteur, erreurs) permettent de différencier les problèmes.

- **Disque dur défaillant** : bruits de cliquetis et démarrage très lent.
- **Disque dur critique** : bruits de cliquetis accompagnés de fichiers inaccessibles.
- **Disque saturé** : démarrage lent, système très lent et espace disque faible.
- **Erreur disque** : écran bleu et message d'erreur lié au disque.

- Problèmes de mémoire vive (RAM)

Cette classe regroupe RAM défectueuse, RAM insuffisante, et problème de mémoire. Tous provoquent des blocages ou des plantages, mais certains sont liés à une défaillance matérielle, d'autres à un manque de capacité.

- **RAM défectueuse** : écran bleu, blocages fréquents et programmes qui plantent.
- **RAM insuffisante** : système très lent, programmes qui plantent et utilisation élevée de la RAM.
- **Problème de mémoire** : écran bleu et blocages fréquents.

- Problèmes périphériques

Plusieurs problèmes : clavier, souris, périphérique USB. Tous affectent l'interaction avec l'ordinateur, mais chaque périphérique a ses propres symptômes.

- **Clavier défectueux** : touches bloquées et clavier qui ne répond pas.
- **Souris défectueuse** : curseur saccadé ou souris non reconnue.
- **Problème avec périphériques USB** : ports USB inactifs ou périphériques non détectés.

- Problèmes carte mère

Carte mère défectueuse et problème carte mère : la carte mère contrôle tous les composants principaux, donc les symptômes peuvent être variés (ordinateur ne démarre pas, bips, ports inactifs).

- **Carte mère défectueuse** : ordinateur qui ne démarre pas, bips anormaux au démarrage, ports USB inactifs.
- **Problème carte mère** : uniquement des bips anormaux.

- Problèmes réseau

Inclut carte réseau défectueuse, problème Wi-Fi, problème configuration réseau, problème connexion Internet. Toutes ces pannes empêchent la connexion, mais la cause peut être matérielle ou logicielle.

- **Carte réseau défectueuse** : pas de connexion Internet, Wi-Fi non détecté, sans problème côté routeur.
- **Problème Wi-Fi** : Wi-Fi non détecté alors que d'autres appareils sont connectés.
- **Problème de configuration réseau** : pas de connexion Internet malgré la présence du réseau.
- **Problème de connexion Internet** : simple absence de connexion.

- Problèmes logiciels

Plusieurs problèmes : infection malware, logiciels malveillants, trop de programmes au démarrage, système lent, fichiers système corrompus. Tous ralentissent l'ordinateur ou provoquent des plantages, mais les causes sont différentes (malware, surcharge, fichiers corrompus).

- **Infection par malware** : système très lent, fenêtres publicitaires intempestives et programmes inconnus.
- **Logiciels malveillants** : fenêtres publicitaires et programmes inconnus.
- **Trop de programmes au démarrage** : démarrage lent, système très lent et nombreux programmes en arrière-plan.
- **Système lent** : ordinateur très lent au démarrage et à l'utilisation.
- **Fichiers système corrompus** : plantages fréquents, messages d'erreur au démarrage et écran bleu.

- Problèmes système d'exploitation

Windows corrompu, problème système d'exploitation, échec de mise à jour. Tous empêchent le bon fonctionnement du système, mais certains empêchent le démarrage, d'autres bloquent les mises à jour.

- **Windows corrompu** : écran bleu, Windows ne démarre pas, fichiers système endommagés.
- **Problème système d'exploitation** : Windows ne démarre pas.
- **Échec de mise à jour** : mise à jour échouée et erreur Windows Update

- Problèmes audios

Pilotes audio corrompus, carte son défectueuse, problème audio, microphone défectueux. Toutes ces pannes affectent le son, mais certaines concernent le matériel, d'autres les pilotes ou périphériques.

- **Pilotes audio corrompus** : absence de son et périphérique audio non détecté.
- **Carte son défectueuse** : absence de son ou son déformé.
- **Problème audio général** : absence de son.
- **Microphone défectueux** : microphone ne fonctionne pas.

2. Détermination des solutions correctives fondées sur la base de connaissances

Cette section présente les solutions associées aux différentes pannes identifiées dans la base de connaissances du système expert. À partir des règles de diagnostic établies, chaque panne est reliée à une action corrective visant à rétablir le fonctionnement normal du système informatique. Les solutions proposées ne constituent pas des procédures exhaustives de réparation, mais des orientations cohérentes et logiquement déduites du diagnostic, permettant de guider l'utilisateur vers les interventions les plus pertinentes. Cette approche souligne le rôle de la base de connaissances non seulement comme outil d'identification des pannes, mais également comme support structuré d'aide à la décision.

- Solution pour problèmes d'alimentation

- **Alimentation défaillante :** vérifier le câble d'alimentation, tester l'ordinateur avec une autre prise électrique ou un autre bloc d'alimentation, et remplacer l'alimentation si nécessaire.
- **Extinction soudaine :** contrôler la température interne, vérifier l'état de l'alimentation et s'assurer de la stabilité de l'alimentation électrique.
- **Bloc d'alimentation faible :** remplacer le bloc d'alimentation par un modèle adapté à la configuration matérielle.

- Solution pour problèmes de refroidissement

- **Surchauffe du processeur :** nettoyer le système de refroidissement, vérifier la pâte thermique et améliorer la ventilation interne.
- **Surchauffe générale :** assurer une bonne circulation de l'air et réduire la charge excessive du système.
- **Ventilateur défectueux :** réparer ou remplacer le ventilateur concerné.

- **Ventilateur bruyant défectueux :** vérifier l'usure ou l'encrassement du ventilateur et envisager son remplacement.
- **Problème de ventilation :** améliorer la ventilation du boîtier et éliminer les obstacles à la circulation de l'air.

- Solution pour problèmes d'affichage

- **Carte graphique défectueuse :** vérifier la connexion de la carte graphique ou procéder à son remplacement.
- **Problème d'affichage :** contrôler les connexions d'affichage et les paramètres graphiques.
- **Écran défectueux :** envisager la réparation ou le remplacement de l'écran.

- Solution pour problèmes disque dur / SSD

- **Disque dur défaillant :** sauvegarder les données et remplacer le disque.
- **Disque dur critique :** effectuer une sauvegarde urgente et remplacer immédiatement le support de stockage.
- **Disque saturé :** libérer de l'espace disque ou étendre la capacité de stockage.
- **Erreur disque :** exécuter des outils de vérification et de réparation du disque.

- Solution pour problèmes de mémoire RAM

- **RAM défectueuse :** tester les barrettes de mémoire et remplacer celles défectueuses.
- **RAM insuffisante :** augmenter la capacité de la mémoire vive.
- **Problème de mémoire :** effectuer des tests mémoire pour identifier la source du dysfonctionnement.

- Solution pour problèmes périphériques

- **Clavier défectueux :** vérifier la connexion ou remplacer le clavier.
- **Souris défectueuse :** tester avec un autre périphérique ou remplacer la souris.
- **Périphérique USB en problème :** tester les ports USB et réinstaller les pilotes associés.

- Solution pour problèmes carte mère

- **Carte mère défectueuse :** effectuer un diagnostic matériel approfondi et envisager le remplacement de la carte mère.
- **Problème de carte mère :** analyser les codes sonores au démarrage afin d'identifier le composant en cause.

- Solution pour problèmes réseau

- **Carte réseau défectueuse :** réinstaller les pilotes réseau ou remplacer la carte.

- **Problème WiFi :** vérifier la configuration WiFi et les paramètres réseau.
- **Problème de configuration réseau :** réinitialiser la configuration réseau.
- **Problème de connexion Internet :** vérifier les paramètres de connexion et la configuration du réseau.

- Solution pour problèmes logiciels

- **Infection par malware :** effectuer une analyse complète du système et supprimer les logiciels malveillants.
- **Logiciels malveillants :** désinstaller les programmes suspects et renforcer la sécurité du système.
- **Trop de programmes au démarrage :** désactiver les programmes inutiles au démarrage.
- **Système lent :** optimiser les ressources système et réduire les processus inutiles.
- **Fichiers système corrompus :** réparer ou restaurer les fichiers système.

- Solution pour problèmes du système d'exploitation

- **Windows corrompu :** réparer ou réinstaller le système d'exploitation.
- **Problème du système d'exploitation :** effectuer une réparation du démarrage.
- **Échec de mise à jour :** corriger les erreurs de mise à jour et relancer le processus.

- Solution pour problèmes audio

- **Pilotes audio corrompus :** réinstaller ou mettre à jour les pilotes audios.
- **Carte son défectueuse :** remplacer ou désactiver la carte son défectueuse.
- **Problème audio :** vérifier la configuration audio du système.
- **Microphone défectueux :** vérifier les paramètres d'entrée audio ou remplacer le microphone.

Après avoir analysé et modélisé les pannes informatiques ainsi que leurs symptômes caractéristiques, nous passons à la phase de réalisation du système expert. Cette étape consiste à transformer les connaissances théoriques formalisées en outil opérationnel capable d'identifier les pannes et de proposer des solutions correctives.

IV. Réalisation du système expert

Cette étape consiste à transformer les connaissances théoriques formalisées en un outil opérationnel capable d'identifier les pannes et de proposer des solutions correctives. La réalisation implique le codage de la base de connaissances et du moteur d'inférence en Prolog, le développement d'une interface utilisateur pour faciliter l'interaction avec le système, ainsi que le choix de technologies complémentaires pour garantir l'efficacité et l'extensibilité du système. L'objectif est de concevoir un système pratique, intuitif et modulable, capable de guider l'utilisateur dans le diagnostic et la résolution des problèmes informatiques.

1. Choix des technologies

- Prolog

Prolog est un langage de programmation déclaratif conçu pour le traitement de la logique et des bases de connaissances. Il est particulièrement adapté aux systèmes experts grâce à sa capacité à représenter des faits, des règles et à effectuer des inférences logiques. Prolog permet de modéliser des relations causales entre événements et symptômes, et d'utiliser un moteur d'inférence pour déduire des conclusions à partir de faits partiels ou incomplets. Son approche basée sur le raisonnement automatique en fait un outil privilégié pour le diagnostic et la prise de décision.



- Tkinter

Tkinter est un module standard de Python qui permet de créer des interfaces graphiques (IHM) de manière rapide et intuitive. Il offre des composants tels que des boutons, des cases à cocher, des menus et des zones de texte, facilitant l'interaction avec l'utilisateur. Python, en tant que langage généraliste, fournit également un environnement flexible pour gérer la logique de l'application et communiquer avec d'autres systèmes, comme un moteur Prolog.



- Communication entre Prolog et tkinter

Pour assurer la transmission des données entre le moteur d'inférence et l'interface utilisateur, des technologies d'interfaçage sont nécessaires. Parmi elles, **PySWIP** (Python – SWI-Prolog interface) permet d'envoyer des faits et des requêtes à Prolog depuis Python, puis de récupérer les résultats pour les afficher dans l'IHM. Ce type de communication garantit une séparation claire entre la logique de raisonnement et la présentation, et permet de rendre le système modulaire et extensible.

2. Implémentation de la base de connaissances en Prolog

La première étape de la réalisation du système expert consiste à traduire la modélisation théorique des pannes et de leurs symptômes en une **base de connaissances opérationnelle** dans Prolog. Cette implémentation repose sur la définition de **faits** et de **règles logiques**, qui permettront au moteur d'inférence de diagnostiquer les pannes à partir des symptômes observés.

La base de connaissances constitue le cœur du système expert. Elle regroupe l'ensemble des faits et des règles permettant de relier les symptômes observés aux pannes correspondantes.

- Gestion dynamique des symptômes

La première étape fonctionnelle du système expert consiste à **collecter et gérer les symptômes** fournis par l'utilisateur. Pour cela, une base dynamique est utilisée.

```
:- dynamic symptome_actif/1.
```

Ce choix permet d'ajouter des symptômes à la volée, de les supprimer entre deux diagnostics et de simuler une interaction utilisateur réelle.

Trois prédicats principaux assurent cette gestion :

```
reset_diagnostic :- retractall(symptome_actif(_)).
```

Ce prédicat réinitialise le diagnostic en supprimant tous les symptômes actifs, garantissant ainsi l'indépendance entre deux sessions.

```
set_symptome(Symptome) :- assertz(symptome_actif(Symptome)).
```
Ce prédicat permet d'enregistrer un symptôme observé par l'utilisateur.

```
a_symptome(Symptome) :- symptome_actif(Symptome).
```
Ce prédicat est utilisé dans toutes les règles de diagnostic pour vérifier la présence d'un symptôme donné.

Cette étape est fondamentale, car **tout le raisonnement du système expert repose sur les symptômes déclarés**.

- Structure des règles de diagnostic

Après la gestion des symptômes, la seconde composante essentielle de la base de connaissances est la définition des règles de diagnostic. Ces règles permettent d'établir un lien logique entre les symptômes observés et les pannes possibles. Chaque panne est décrite par un ensemble de symptômes caractéristiques, et une règle Prolog permet d'identifier la panne lorsque ces symptômes sont présents.

Afin d'illustrer le fonctionnement des règles de diagnostic, nous présentons ci-dessous un exemple représentatif extrait de la base de connaissances. Il s'agit de la règle permettant de diagnostiquer une panne de carte graphique défectueuse.

```
panne(carte_graphique_defectueuse) :-
    a_symptome(ecran_noir_demarrage),
    a_symptome(ordinateur_demarre_sons),
    \+ a_symptome(cable_ecran_defectueux).
```

Cette règle indique que la panne *carte_graphique_defectueuse* est déduite lorsque l'écran reste noir au démarrage, que l'ordinateur semble fonctionner normalement (sons de démarrage présents), et qu'aucun problème lié au câble d'affichage n'a été détecté. L'utilisation de la négation logique (\+) permet d'exclure une cause alternative simple, ce qui améliore la fiabilité du diagnostic.

De même manière les autres pannes sont modélisées sous forme de règles Prolog à l'aide du prédicat panne et sont organisées en plusieurs classes de pannes (alimentation, affichage, stockage, mémoire, réseau, logiciels, etc.), ce qui facilite la structuration de la base de connaissances. Chaque règle associe une panne spécifique à un ensemble cohérent de symptômes.

- Gestion des pannes partielles

Dans un contexte réel, l'utilisateur ne dispose pas toujours de tous les symptômes nécessaires à un diagnostic précis. Afin de rendre le système plus robuste, des règles de pannes partielles ont été introduites.

```
panne_partielle(probleme_alimentation) :-
    a_symptome(ordinateur_ne_demarre_pas).
```

Ces règles permettent au système de proposer des hypothèses de pannes même lorsque les informations sont incomplètes, évitant ainsi un diagnostic vide et améliorant l'expérience utilisateur.

- **Encodage des solutions correctives**

Chaque panne identifiée est associée à une solution descriptive à l'aide du prédicat solution. Les solutions fournissent non seulement un diagnostic, mais également des actions correctives structurées et un niveau de difficulté.

```
solution(alimentation_defaillante,
'PROBLÈME D\'ALIMENTATION ÉLECTRIQUE

✓ DIAGNOSTIC:
  Votre ordinateur ne reçoit pas d\'alimentation électrique.
  Les composants ne sont pas alimentés correctement.

🔧 SOLUTIONS:
  1. Vérifiez que le câble d\'alimentation est bien branché
  2. Testez avec un autre câble d\'alimentation
  3. Vérifiez l\'interrupteur sur le bloc d\'alimentation (position ON)
  4. Testez la prise électrique avec un autre appareil
  5. Remplacez le bloc d\'alimentation si défectueux
⚠  Niveau difficulté: Moyen - Peut nécessiter un technicien').
```

- Moteur d'inférence et logique de raisonnement

Le raisonnement du système expert repose sur le prédicat run_diagnostic.

```
run_diagnostic(Panne, Solution) :-
    panne(Panne),
    !,
    solution(Panne, Solution).
```

Le moteur utilise le chaînage arrière, mécanisme natif de Prolog, pour rechercher une panne correspondant aux symptômes actifs. En l'absence de correspondance exacte, il se replie sur les pannes partielles, puis retourne un diagnostic inconclusif si aucune règle ne s'applique. L'opérateur de coupure (!) permet de retenir un diagnostic unique et cohérent.

- Fonctions utilitaires

Des prédicats utilitaires complètent le système afin de faciliter le suivi et l'interaction :

```
liste_symptomes_actifs(Liste).
compte_symptomes(Nombre).
a_des_symptomes.
toutes_pannes_possibles(ListePannes).
```

Ils permettent notamment de vérifier les symptômes actifs et d'assister l'intégration avec l'interface graphique.

3. Interface IHM avec Tkinter

L'interface graphique constitue le point d'interaction entre l'utilisateur et le système expert. Son rôle est de permettre la saisie intuitive des symptômes, de déclencher le raisonnement du moteur d'inférence Prolog et de présenter le diagnostic et les solutions de manière lisible.

- Initialisation et connexion au moteur Prolog

Le lien entre Python et Prolog est assuré par PySWIP, une interface qui permet d'exécuter des requêtes Prolog depuis Python. La base de connaissances `diagnostic_pc.pl` est chargée au démarrage :

```
from pyswip import Prolog

prolog = Prolog()
try:
    prolog.consult("diagnostic_pc.pl")
    print("✓ Prolog chargé")
except Exception as e:
    print(f"✗ Erreur: {e}")
```

Cette étape garantit que toutes les **règles et solutions définies en Prolog** sont accessibles à l'application Python.

- Organisation des symptômes

Les symptômes sont classés en catégories thématiques (démarrage, température, performance, disque, affichage, réseau, audio, virus…) pour faciliter la navigation. Chaque symptôme est représenté par un code interne (utilisé pour Prolog) et une description lisible pour l'utilisateur :

```
SYMPTOMES = {
    "💻 DÉMARRAGE": [
        ("ordinateur_ne_demarre_pas", "L'ordinateur ne démarre pas"),
        ("aucun_bruit_ventilateur", "Aucun bruit de ventilateur"),
        ("ecran_noir_demarrage", "Écran noir au démarrage"),
    ],
```

```
    "📎 TEMPÉRATURE": [
        ("ordinateur_tres_chaud", "Ordinateur très chaud"),
        ("surchauffe", "Surchauffe"),
    ],
    ...
}
```

Cette structure facilite l'**automatisation de l'affichage** et la collecte des informations.

- Création de la fenêtre principale

La classe DiagnosticApp encapsule toute la logique de l'application :

```
class DiagnosticApp:
    def __init__(self, root):
        self.root = root
        self.root.title("Diagnostic PC - Système Expert")
        self.root.geometry("900x650")
        self.vars = {}
        self.creer_interface()
```

Elle initialise la fenêtre principale, définit sa taille et prépare un dictionnaire self.vars pour stocker l'état des cases à cocher des symptômes.

- Génération dynamique des catégories et symptômes

Pour chaque catégorie, un bouton pliable permet d'afficher ou masquer les symptômes :

```
btn = Button(btn_frame, text=f"▶  {categorie}",
command=make_toggle(symp_frame, btn, categorie))
```

Chaque symptôme est associé à une variable booléenne qui reflète l'état de sélection dans l'interface et sert ensuite pour le diagnostic Prolog :

```
var = BooleanVar()
Checkbutton(symp_frame, text=desc, variable=var).pack(fill=X)
self.vars[code] = var
```

- Collecte des symptômes et diagnostic

Lorsqu'un utilisateur clique sur "Diagnostiquer", le programme réinitialise la base dynamique de Prolog (reset_diagnostic) pour éviter les conflits avec les précédents diagnostics, parcourt toutes les cases à cocher pour collecter les symptômes sélectionnés, envoie chaque symptôme actif au moteur Prolog via set_symptome et lance le diagnostic avec run_diagnostic(P, S).

```
for code, var in self.vars.items():
    if var.get():
        list(prolog.query(f"set_symptome({code})"))

resultats = list(prolog.query("run_diagnostic(P, S)"))
```

Cette approche permet de reproduire le raisonnement logique défini en Prolog directement depuis l'interface graphique.

- Affichage du résultat

Le diagnostic est présenté dans une fenêtre secondaire avec le nom de la panne détectée,la solution complète avec les étapes correctives et la liste des symptômes analysés.

```
text.insert(END, f"PANNE: {panne}\n")
text.insert(END, solution + "\n\n")
for i, s in enumerate(symptomes, 1):
    text.insert(END, f"{i}. {s}\n")
```

Cette présentation structurée rend le système explicatif et pédagogique, en permettant à l'utilisateur de comprendre la logique derrière le diagnostic.

- Réinitialisation

Un bouton "Réinitialiser" permet de désélectionner tous les symptômes, garantissant une nouvelle session propre :

```
for var in self.vars.values():
    var.set(False)
```

Cette fonctionnalité assure que chaque diagnostic est indépendant et reflète fidèlement l'état actuel de la machine.

V. Résultats obtenus

Après l'implémentation complète du système expert, incluant la base de connaissances en Prolog et l'interface utilisateur en Tkinter, nous avons évalué son fonctionnement à travers différents scénarios de diagnostic. L'objectif de cette partie est de présenter les résultats observés, de montrer la capacité du système à identifier les pannes et à fournir des solutions correctives, ainsi que d'évaluer son efficacité et sa robustesse**.**

1. Fonctionnement général

Le système expert fonctionne selon le processus suivant :

1. **Sélection des symptômes :** l'utilisateur coche les symptômes observés via l'IHM Tkinter, classés par catégories (démarrage, température, disque, affichage, etc.).
2. **Transmission à Prolog** : chaque symptôme sélectionné est envoyé au moteur Prolog, qui met à jour la base dynamique des faits.
3. **Diagnostic automatique :** le moteur d'inférence utilise le chaînage arrière pour déterminer la panne la plus probable à partir des symptômes déclarés.
4. **Affichage de la solution :** le système fournit non seulement le diagnostic, mais également des actions correctives détaillées, accompagnées d'un niveau de difficulté.

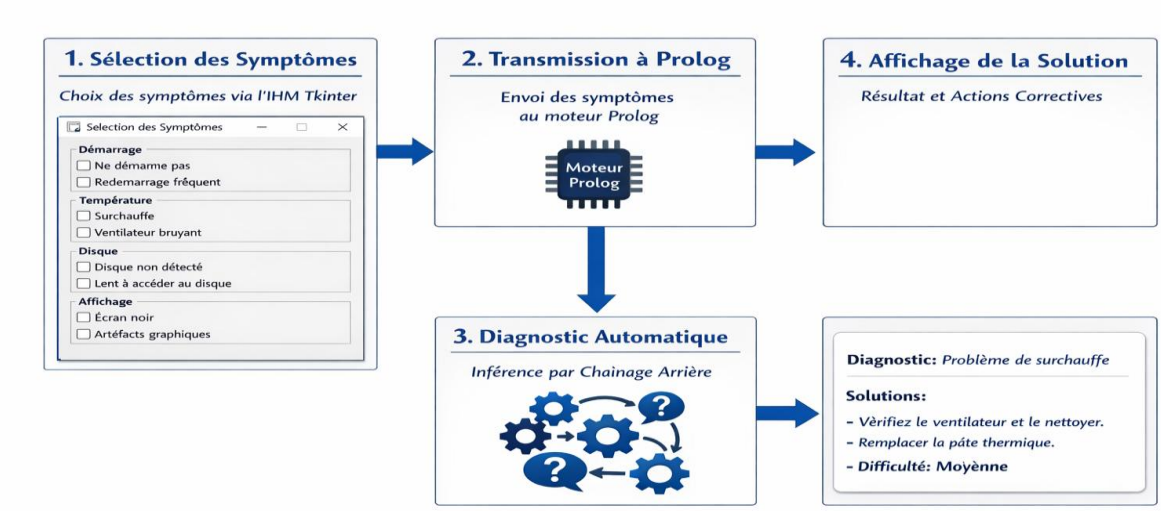


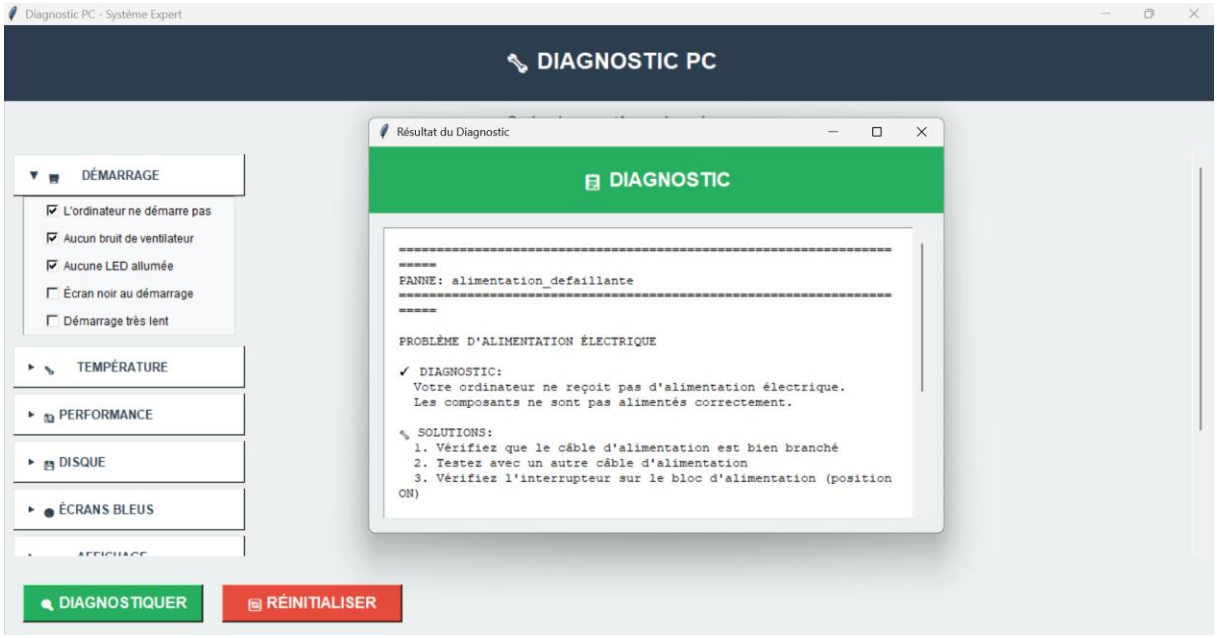
Figure 2 Illustration du fonctionnement de notre système expert

2. Exemples de résultats

Plusieurs tests ont été réalisés avec des symptômes simulant différentes pannes. Par exemple :

- **Cas 1 : Alimentation défaillante**

  Symptômes sélectionnés : ordinateur ne démarre pas, aucun bruit de ventilateur, aucune LED allumée.

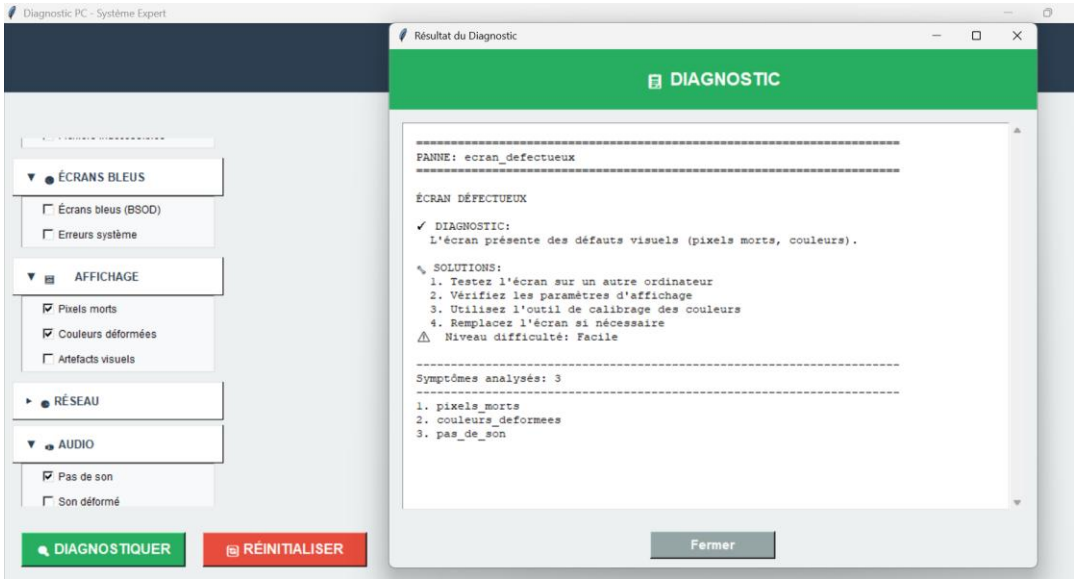


Diagnostic Prolog : alimentation défaillante

Solution affichée : vérification du câble, test du bloc d'alimentation, remplacement si nécessaire.

- **Cas 2 : Ecran défectueux**

Symptômes sélectionnés : pixels morts, couleurs déformées



Diagnostic Prolog : écran défectueux

Solution affichée : Testez l'écran sur un autre ordinateur, vérifiez les paramètres d'affichage.

Dans chaque situation, le système a identifié correctement la panne et a proposé des solutions cohérentes avec la pratique réelle, démontrant la pertinence de la base de connaissances et des règles de diagnostic.

3. Performance

Le système expert développé présente plusieurs qualités notables. Sa rapidité permet d'obtenir un diagnostic en quelques secondes, même lorsque l'utilisateur sélectionne un grand nombre de symptômes. Sa robustesse est assurée par l'intégration de règles de pannes partielles, capables de générer des hypothèses même lorsque certaines informations sont absentes, ce qui reflète une approche proche du raisonnement humain. De plus, le système se distingue par son explicabilité : chaque diagnostic est accompagné d'une justification claire et d'actions correctives détaillées, facilitant la compréhension et la prise de décision pour l'utilisateur. Enfin, l'extensibilité de la base de connaissances permet d'ajouter de nouvelles pannes ou symptômes sans modifier l'interface graphique, garantissant une évolution facile et modulable du système.

VI. Conclusion

Le projet de système expert pour le diagnostic des pannes informatiques a permis de transformer une modélisation théorique des problèmes matériels et logiciels en un outil opérationnel, capable de fournir des diagnostics rapides, précis et explicables. La combinaison de Prolog pour le raisonnement logique et de Tkinter pour l'interface utilisateur a permis de créer un système à la fois robuste, interactif et modulable.

L'implémentation a mis en évidence plusieurs points forts : la gestion dynamique des symptômes, la structuration des règles de diagnostic en classes cohérentes, l'intégration de pannes partielles pour pallier l'absence d'informations complètes, et l'association de solutions correctives détaillées pour guider l'utilisateur. Ces caractéristiques garantissent une expérience utilisateur fluide, tout en offrant une base extensible pour intégrer de nouvelles pannes ou fonctionnalités.

Cependant, certaines limites subsistent, notamment la dépendance à la saisie manuelle des symptômes et l'absence d'adaptation automatique aux nouvelles défaillances émergentes. Des améliorations futures pourraient inclure l'intégration de modules d'apprentissage automatique, la connexion à des bases de données externes, ou des fonctionnalités de diagnostic en temps réel, afin de rendre le système encore plus intelligent et autonome.

En somme, ce projet illustre concrètement comment les principes des systèmes experts peuvent être appliqués à un cas réel, offrant un outil pratique, pédagogique et évolutif pour le diagnostic informatique.

Webographie :

https://www.docteurordinateur.com/les-12-pannes-les-plus-courantes-d-un-ordinateur.html

https://python.doctor/page-tkinter-interface-graphique-python-tutoriel

https://medium.com/@alihassanshahid/implementing-prolog-in-python-using-vs-code-and-colab-e0f5c9adce02

https://ksvi.mff.cuni.cz/~dingle/2019/npp/prolog_reference.html

https://jt-informatique.fr/conseils/panne-dordinateur-frequente/